



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

LLM Benchmark project Report

Professors:

Michele Lombardi

Allegra De Filippo

Students:

Simone Migliaccio (matr.:
0001159303)

Simone Spezia (matr.:
0001167605)

Alberto Varini (matr.:
0001189363)

1 Introduction

The main goal of this project is to provide a general benchmark to evaluate different LLMs performances under different types of planning domains to better understand in advance what are the planning capabilities of a given LLM. In the following sections the main features of this projects are going to be shown.

2 PDDL Problems Classification

The first thing to do since we wanted a trustable benchmark and since a lot of PDDL models were already provided as reference material was to classify the problems using a numerical difficulty score. We used a numerical score because it is intuitively understandable by anyone who will ever want to compare the performance of LLMs using these models. Another classification implemented consists into separating problems in different categories based on the type of reasoning they request. To compute the scores the code can be found in the file `score_domains_complexity.py`

2.1 Difficulty Classification

2.1.1 Instances scores

The complexity of a single problem instance is calculated using a weighted logarithmic combination of four key metrics. The use of the logarithm prevents any single feature from dominating the score excessively.

The **Raw Score** is defined as:

$$Raw_score = 0.45 \cdot \log(1 + A) + 0.20 \cdot \log(1 + G) + 0.20 \cdot \log(1 + N) + 0.15 \cdot T \quad (1)$$

Where:

- **A (Actions):** Number of feasible grounded actions. This is estimated by unifying action parameters with objects that satisfy static preconditions in the initial state.
- **G (Goals):** Total number of atomic conditions in the goal state.
- **N (Numeric Score):** A composite value representing numeric complexity:

$$N = E_{num} + 2 \cdot P_{num} + 3 \cdot G_{num} + M \quad (2)$$

where E_{num} are numeric effects, P_{num} numeric preconditions, G_{num} numeric goal conditions, and M is a binary flag for the presence of a metric.

- **T (Topology Score):** Evaluates the spatial/relational complexity based on predicates like *adj* or *connected*:

$$T = \log(1 + Nodes) + \log(1 + Edges) + Density \quad (3)$$

Obviously a different weight could be assigned to each different part of the computation of raw scores depending on the operator that is doing the tests in order to highlight more for example the number of action present in the instances and so on.

Finally, an **Instance Score (0-100)** is calculated by normalizing the raw score relative to the

minimum and maximum scores found within the same domain. The results can be found in the folder `domains_complexity`.

Note that also some additional statistics for each domain are computed based on the results obtained from the instances, for example it could be of interest evaluating the mean and the median of the scores obtain form the instances of a single domain.

2.1.2 Domain scores

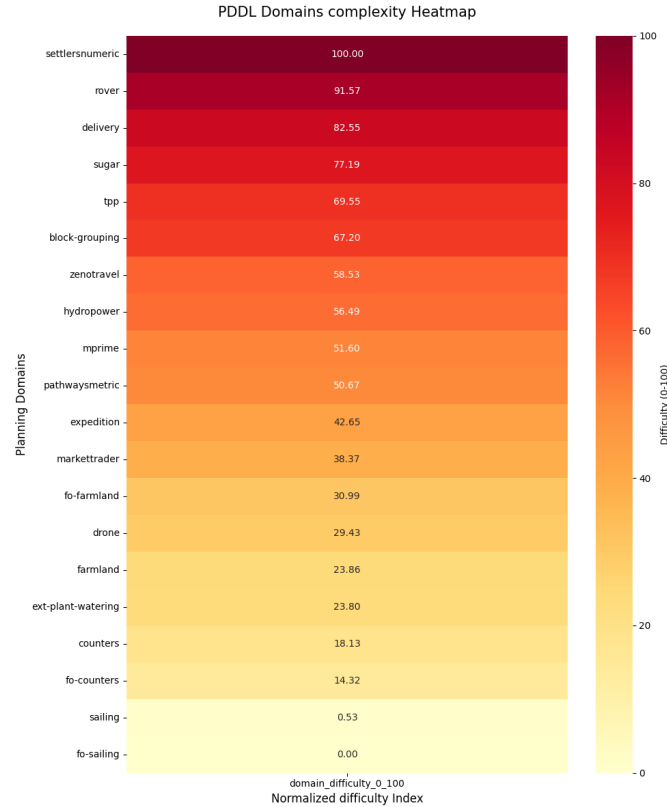
The overall difficulty of a domain is not just the average of its instances, but a reflection of its total state-space and constraint complexity. The **Domain Difficulty (0-100)** is computed by:

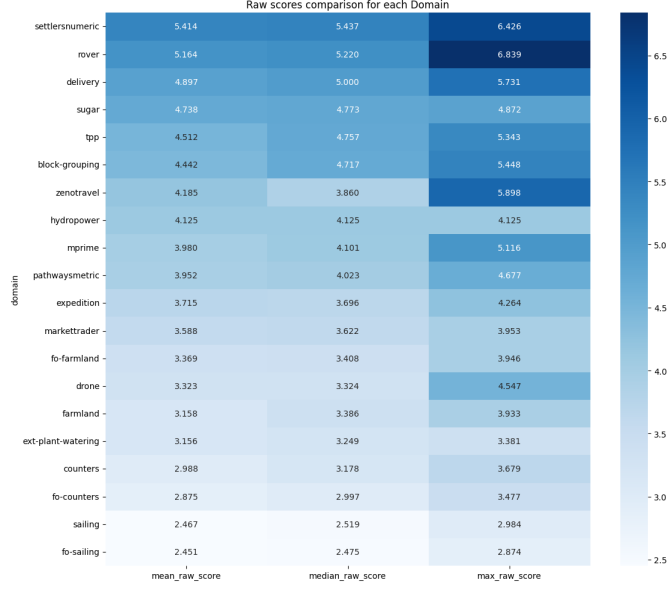
1. Summing the S_{raw} of all instances belonging to the domain to obtain a *Total Raw Score*.
2. Normalizing this total across all tested domains in the benchmark.

This approach ensures that a domain with many complex instances is ranked higher than one with few simple instances, note that obviously the easiest problems gets a domain difficulty equal to 0 (in this case is the "fo-sailing" problem) and the problem with the highest difficulty has domain difficulty equal to 100, (in our case the "settlersnumeric" problem).

2.1.3 Results obtained

In order to obtain a visual representation of the classified problems, we produced heatmaps for both the overall normalized domain scores and the raw scores.





2.2 Reasoning Classification

Here the classification based on the type of reasoning these problems require the LLM to employ is shown. We decided to split the reasoning types into 5 categories:

- **Spatial:** These problems are typically about objects that have to be moved in a 2D (or more dimensional) spaceframe.
- **Numerical Optimization:** These problems require from the LLM an extensive capacity to deal with numerical optimization problems, such as managing complex cost functions and resource constraints.
- **Temporal Planning:** In this kind of problem it is required from the LLM to be able to reach the goal defining an order between objects having timing constraints and relations.
- **Logistic:** In this kind of problems it is required to reach the goal by planning some kind of goods transportation and agents coordination.
- **Logic:** These are abstract puzzles where the state space and actions represent symbolic or mathematical transformations rather than physical interactions. They test the model's ability to reason over purely formal rules without relying on world knowledge.

In the following table the classification of the 20 planning problems is shown, note that in this case all the work have been done by hand and that the problems in the table are ordered from hardest to easiest following the previous difficulty classification.

PDDL Problem	Reasoning Category
Settlersnumeric	Numerical Optimization
Rover	Logistic
Delivery	Logistic
Sugar	Numerical Optimization
Tpp	Numerical Optimization
Block-Grouping	Spatial
Zenotravel	Logistic
Hydropower	Temporal Planning
Mprime	Logic
Pathwaysmetric	Numerical Optimization
Expedition	Logistic
Markettrader	Numerical Optimization
Fo-farmland	Numerical Optimization
Drone	Spatial
Farmland	Numerical Optimization
Ext-Plant-Watering	Spatial
Counters	Numerical Optimization
Fo-Counters	Numerical Optimization
Sailing	Spatial
Fo-Sailing	Spatial

Table 1: Risultati di classificazione per le 20 istanze, con parametri centrati.

2.3 Problems chosen

By combining instance-level and domain-level metrics, we can categorize problems into three tiers: **Easy**, **Medium**, and **Hard**. This classification is fundamental to evaluate if an LLM’s success rate drops predictably as the complexity of the grounded action space or numeric constraints increases.

3 Parser and LLMs Models Implemented

4 Validator

5 Output Comparison

6 Risultati

Table 2: Performance dei modelli nel dominio Farmland

Modello	Success Rate (%)	Lunghezza Media Piano	Iterazioni Medie
GPT-4o	85.0	12.4	1.2
Gemini 1.5 Pro	88.5	11.8	1.1
Llama 3 70B	72.0	14.2	2.4

7 Conclusioni

Discussione finale sui risultati ottenuti e suggerimenti per future estensioni del benchmark, come l'aggiunta di domini temporali.